From ERD to Relational Model

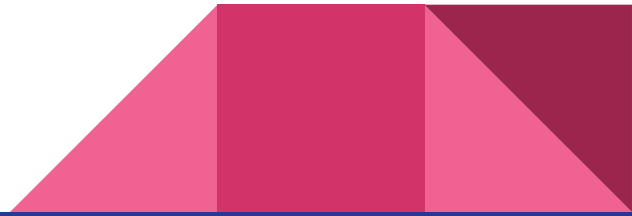
Limits & Transformations

Relational model - Constraint types

To store data in databases, it must first be adapted to the structures imposed by the model being used.

The relational model imposes two types of constraints:

- **Inherent constraints**: These are constraints that are intrinsic to the relational model itself, such as the requirement that each relation (table) has a unique name and that each tuple (row) contains the same number of attributes (columns).
- **Semantic constraints**: These arise from the need to represent the real world. They are applied by users when designing a database schema.
    - **PRIMARY KEY** - Key used to identify each register
    - **UNIQUE** - No duplicates allowed
    - **NOT NULL** - No NULL values allowed
    - **FOREIGN KEY** - Referential integrity



Relational model - Constraints - Referential integrity

Regarding referential integrity, it is also important to keep in mind that it imposes limitations on certain operations:

- Cascading deletion or update (**CASCADE**): If a record or tuple referenced by another table is deleted or updated, the corresponding tuples in the other table are also deleted or updated, respectively.

- Restricted deletion or update (**RESTRICT**): Tuples in the referenced table (where the primary key is used as a foreign key in another table) cannot be deleted or updated as long as there are related values in the referencing table.

- Deletion or update by setting the value to NULL or to a default value (**SET NULL** and **SET DEFAULT**, respectively): If a record is deleted or updated in the referenced table, the foreign key in the referencing table is set to NULL or to a default value, respectively.

Relational model - Constraints - Triggers

Triggers are a type of **procedural** constraint (similar to a sequential program).

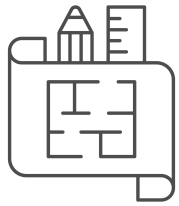
When a specific condition is met, the **user defines a series of actions** to be executed.

```
CREATE TRIGGER check_salary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
        IF NEW.salary < 0 THEN
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Salary cannot be negative';
        END IF;
END;
```

Relational model - Transformations - Basic rules

There are several basic rules for transforming an Entity-Relationship Diagram (ERD) into a logical model, such as a Relational model schema:

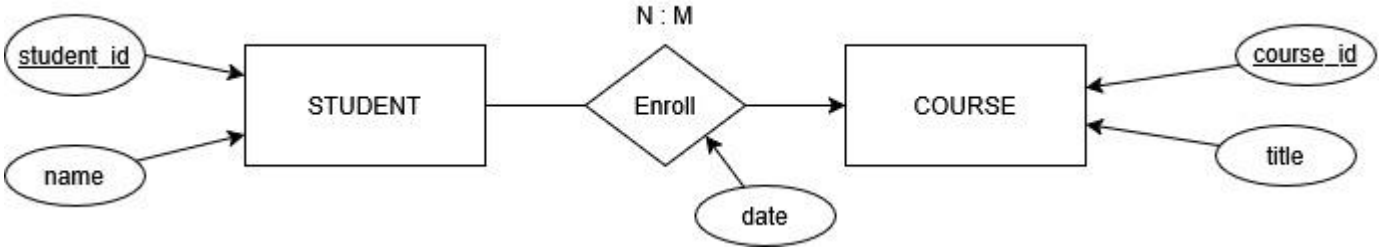
- **Entities** are transformed into **tables**.
- **Attributes** are transformed into **columns** within their corresponding tables.
- **Primary keys must be preserved**: the primary key of each entity becomes the primary key of its corresponding table.



Relational model - Transformations - N:M

All **N:M** interrelationships **always translate to a new table**.

The **primary key** in this new table is *usually* **composed** of the primary keys of the two related tables, which become **two foreign keys** referencing their respective tables.



STUDENT(student_id, name)
COURSE(course_id, title)
ENROLLMENT(student_id(FK), course_id(FK), enrollment_date)

Relational model - Transformations - 1:N

For a 1:N relationship, there are **two** possible transformations:

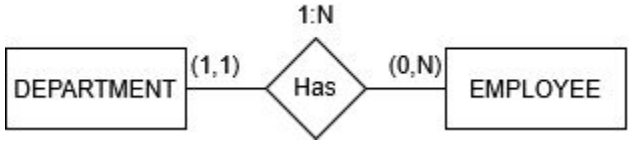
- **Propagate the key:** The **standard procedure**. Always applied when the **N side (many side) participation is mandatory**, with cardinality (1,N). The primary key of the entity on the **one** side is added as a foreign key to the entity on the **many** side. Any attributes belonging to the relationship are also transferred.
    - Requires **NOT NULL** Foreign Keys when N side is mandatory.

- **Transform the relationship into a table:** This is used as an *alternative* when **N side participation is optional**, with cardinality (0,N). As with an N:M relationship, a new table is created. The primary key of the new table is the primary key of the entity on the **many** side (also a Foreign Key). Any attributes belonging to the relationship are added to the new table.
    - Avoids NULL values in Foreign Keys

Relational model - Transformations - 1:N - Participation

Example 1: DEPARTMENT – (1:N) – EMPLOYEE (Employee entity is the “many side” or N side)

- Each Department may have zero or more Employees (**optional participation on the 1-side**).
- Each Employee must belong to exactly one Department (**mandatory participation on the N-side**).



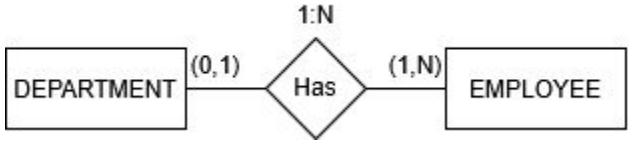
Relational model transformation: **key propagation**. The *DepartmentID* foreign key is placed in the N-side (EMPLOYEE) table and **must be NOT NULL because every employee must belong to a department**.

DEPARTMENT (DepartmentID, Name)
EMPLOYEE (EmployeeID, Name, DepartmentID (**NOT NULL**))
→ {DepartmentID} REFERENCES DEPARTMENT

Relational model - Transformations - 1:N - Participation

Example 2: DEPARTMENT – (1:N) – EMPLOYEE (Employee entity is the “many side” or N side)

- Each Department must have at least one Employee (**mandatory participation on the 1-side**).
- Each Employee may belong to zero or one Department (**optional participation on the N-side**).



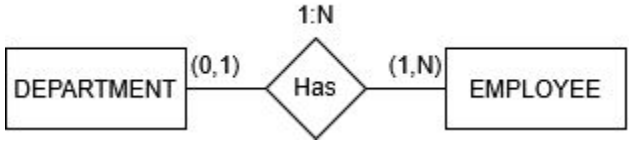
Relational model transformation: key propagation: Here the *DepartmentID* primary key is placed in the N-side (EMPLOYEE) as a Foreign Key. It can be NULL because an employee may not belong to a department.

DEPARTMENT (DepartmentID, Name)
EMPLOYEE (EmployeeID, Name, DepartmentID) → {DepartmentID} REFERENCES DEPARTMENT

Relational model - Transformations - 1:N - Participation

Example 2: DEPARTMENT – (1:N) – EMPLOYEE (Employee entity is the “many side” or N side)

- Each Department must have at least one Employee (**mandatory participation on the 1-side**).
- Each Employee may belong to zero or one Department (**optional participation on the N-side**).



Alternative relational model transformation: create a **new relationship table**. In this case, *EmployeeID* is the primary key of the new table HAS because each employee can belong to at most one department. This transformation avoids having NULL values in a Foreign Key.

DEPARTMENT (DepartmentID, Name)
EMPLOYEE (EmployeeID, Name)
HAS (EmployeeID, DepartmentID)
→ {EmployeeID} REFERENCES EMPLOYEE
→ {DepartmentID} REFERENCES DEPARTMENT

Relational model - Transformations - 1:1

A 1:1 relationship has **three** possible transformations:

- **Merge into a single table:** When **both entities have cardinality (1,1) - both sides are mandatory**, every instance of one entity must be related to exactly one instance of the other, and vice versa. The two entities can therefore be merged into a single table.
- **Transform the relationship into a new table:** When both entities have cardinality **(0,1)** - **participation is optional on both sides**. A separate table can be created to represent the relationship, especially when the relationship has its own attributes.
- **Propagate the key:** When one entity has cardinality **(1,1)** and the other has **(0,1) - one side is mandatory and the other optional.** the primary key of the (1,1) entity is added as a foreign key to the (0,1) entity. This ensures that **every instance of the mandatory entity is associated with an instance of the optional entity**, while allowing the optional entity to have no corresponding instance.
    - To enforce the 1:1 relation, Foreign Key also needs to be **UNIQUE**.

Relational model - Ternary (or more) Transformations

Whenever there are three or more entities with an interrelationship established, there will be only one possible outcome when transforming from ERD to relational model: **the interrelationship will become a new table**.

In ternary transformations, **the primary keys of all three participating entities are included in the new table as foreign keys**.

Meanwhile, the **primary key depends on each individual case** and is *usually* formed by combining the primary keys of the three participating entities. However, depending on the cardinality constraints, a smaller combination of these keys may be sufficient.

Relational model - Ternary Transformations - Primary Key

The **primary key includes the foreign keys corresponding to the N** sides (many-sides).

Cardinality	Primary Key
N:N:N	**All Foreign Keys** combine to form the Primary Key
1:N:N	**Foreign Keys from both N** sides are combined to form the Primary Key
1:1:N	**Only the Foreign Key from the N** side as Primary Key
1:1:1	**Any one of the Foreign Keys** can be the Primary Key

Relational model - Reflexive/Recursive Transformations - N:M

Although the interrelationship is between itself, this transformation can be treated as a binary interrelationship; a relation between two entities.

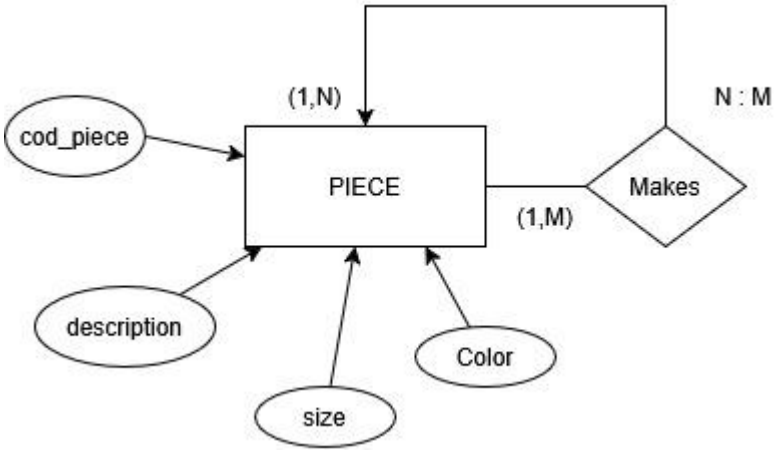
Therefore, the outcome will depend on the cardinality and participation.

For **N:M** interrelationships, the transformation **always results in a new table**.

The new table will contain **TWO foreign keys referencing the primary key** from the entity, and these two attributes **combined will form primary key** of the new table.



Relational model - Reflexive/Recursive Transformations - N:M



PIECE - MAKES - PIECE

A piece is made up of many pieces, which in turn are made up of other pieces.

PIECE (CodPiece, Description, Size, Color)

MAKES (CodPiece, CodPiece_Makes)
WHERE {CodPiece} REFERENCES PIECE and
WHERE {CodPiece_Makes} REFERENCES PIECE

Relational model - Reflexive/Recursive Transformations - 1:N and 1:1

For recursive 1:N and 1:1 relationships, the **standard transformation is to propagate the primary key as a self-referencing foreign key** in the same table.

Although a new, separate relationship table is also possible, **key propagation is generally preferred** because it is simpler and **can enforce mandatory participation constraints more directly**.



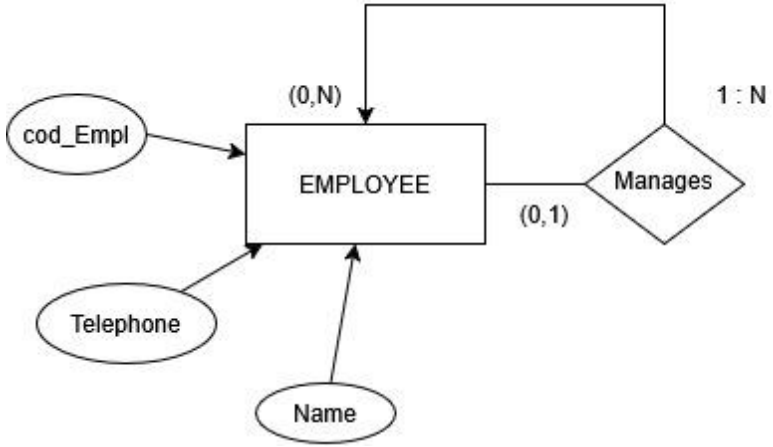
Relational model - Reflexive/Recursive Transformations - 1:N

For a 1:N relationship, there are **two** possible transformations, depending on participation:

- **Propagate the key:** Add a self-referencing **foreign key** to the same table, **referencing its primary key**. This is the standard approach when each instance on the N-side can reference at most one instance on the 1-side.
    - If participation on the N-side is **mandatory**, the **foreign key must be NOT NULL**.

- **Transform the relationship into a table:** Create a **new table containing two foreign keys** referencing the primary key of the same entity table, representing the two roles in the relationship. The **primary key is the foreign key corresponding to the N-side**. This **avoids having NULL** values in the entity table, since only existing relationships are stored.
    - **ONLY FOR OPTIONAL**: A new table does **not guarantee mandatory** participation.

Relational model - Reflexive/Recursive Transformations - 1:N



EMPLOYEE (<u>CodEmpl</u>, Name, Telephone, CodManager)
WHERE {CodManager} REFERENCES EMPLOYEE

EMPLOYEE (CodEmpl, Name, Telephone)

MANAGES (<u>CodEmpl</u>, CodManager)
WHERE {CodEmpl} REFERENCES EMPLOYEE and
WHERE {CodManager} REFERENCES EMPLOYEE

EMPLOYEE - MANAGES - EMPLOYEE

An employee **can** manage zero or many employees, while each employee **can** have at most one manager. (Participation is optional)

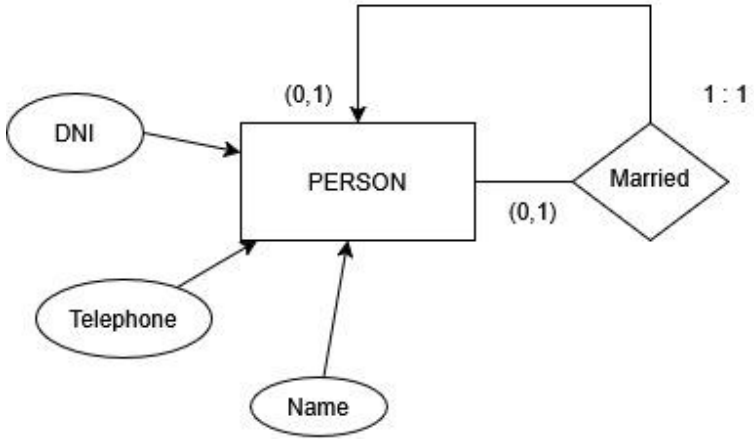
Relational model - Reflexive/Recursive Transformations - 1:1

For a 1:1 relationship, similar to 1:N, there are **two** possible transformations, depending on participation:

- **Propagate the key:** Add a self-referencing **foreign key** to the same table, **referencing its own primary key**.
    - Specify **NOT NULL** to enforce **mandatory** participation for the role represented by the foreign key.
    - Specify **UNIQUE** to enforce that each entity is related to **at most one** other entity.

- **Transform the relationship into a table:** Create a **new table containing two foreign keys**, both referencing the primary key of the same entity table and **representing the two roles** in the relationship. The **combination of both foreign keys can serve as the primary key**. This option avoids NULLs in the entity table, since only existing relationships are stored.
    - Specify **UNIQUE** constraints **on each foreign key to enforce 1:1** cardinality.
    - **ONLY FOR OPTIONAL**: A new table does **not guarantee mandatory** participation.

Relational model - Reflexive/Recursive Transformations - 1:1



PERSON - MARRIED - PERSON

A person **can** be married to at most one other person, and the relationship is **optional** for both individuals.

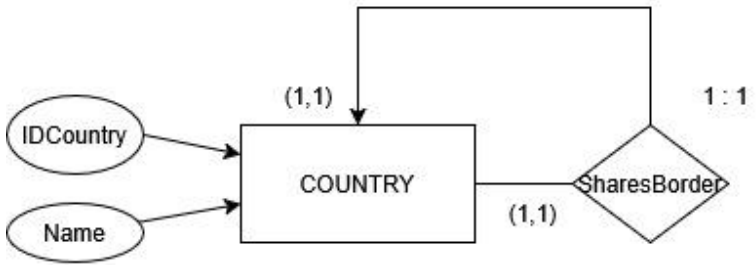
PERSON (DNI, Name, Telephone, DNIPartner (**UNIQUE**))
WHERE {DNIPartner} REFERENCES PERSON

PERSON (DNI, Name, Telephone)

MARRIED (DNI (**UNIQUE**), DNIPartner (**UNIQUE**))
WHERE {DNI} REFERENCES PERSON and
WHERE {DNIPartner} REFERENCES PERSON

Specify UNIQUE to ensure that each person can be associated with **at most** one partner.

Relational model - Reflexive/Recursive Transformations - 1:1



COUNTRY (IDCountry, Name, IDNeighbour (**NOT NULL, UNIQUE**)) WHERE {IDNeighbour } REFERENCES COUNTRY

COUNTRY (IDCountry, Name)

BORDER (IDCountry (**NOT NULL, UNIQUE**), IDNeighbour (**NOT NULL, UNIQUE**))
WHERE {IDCountry} REFERENCES COUNTRY and
WHERE {IDNeighbour} REFERENCES COUNTRY

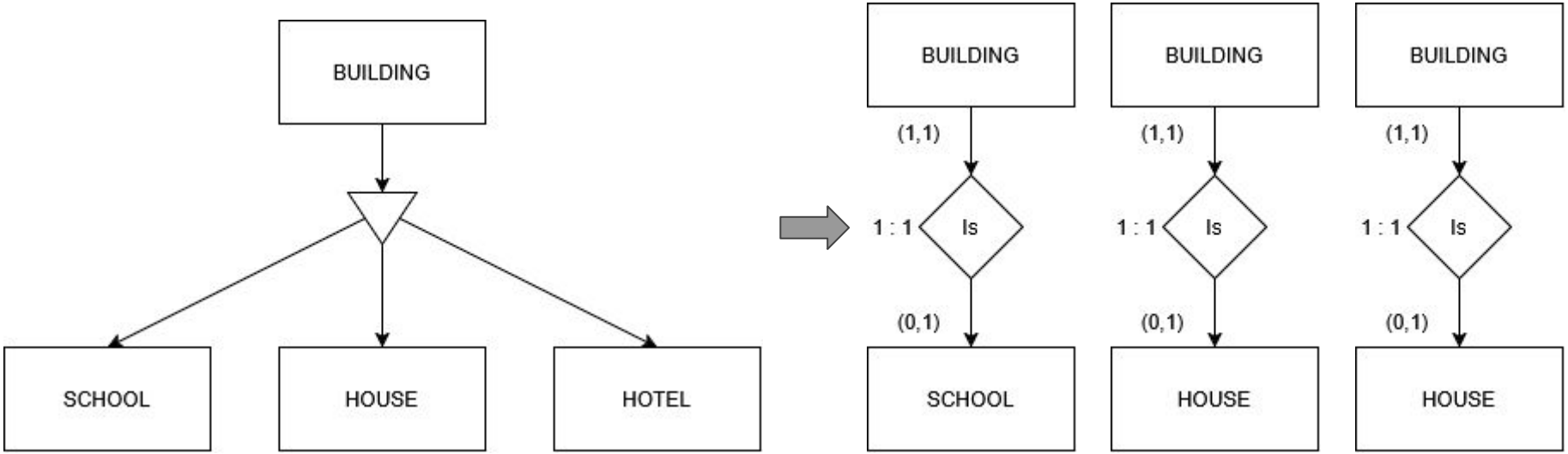
- NOT NULL ensures that **every country is assigned a neighbour**ing country (preventing NULL values).
- Having a **new table** does **NOT guarantee mandatory** participation for each Person.

COUNTRY - shares border - COUNTRY

Participation is **mandatory** because every country **must** share a border with exactly one other country.

Relational model - From Inheritance to Binary to Relational model

Inheritance can be considered as multiple binary relationships: replace the inheritance hierarchy with binary relationships between the superior class and each sub-class, using a 1:1 relationship.



Relational model - Inheritance Transformation

Treating inheritance as binary 1:1 relationships with one side being optional (subclass) and the other mandatory (superior class) means that the standard transformation will be **key propagation**.

Create a table for the superclass entity and a separate table for each subclass entities. **The superclass primary key is propagated to each subclass table as both a primary key and foreign key**, linking subclass instances to their superclass.

BUILDING (IDBuilding)
SCHOOL (IDBuilding) → {IDBuilding} REFERENCES BUILDING
HOUSE (IDBuilding) → {IDBuilding} REFERENCES BUILDING
HOTEL (IDBuilding) → {IDBuilding} REFERENCES BUILDING

Degree	Cardinality	Transformations
Unary	1:1	- Both sides mandatory: **Key propagation** self-referencing (requires **NOT NULL** and **UNIQUE**). - Both sides optional: **Key propagation** self-referencing (requires **UNIQUE**). **Alternative: New table** to avoid NULL FKs (requires **UNIQUE**). - One optional, One mandatory: **Key propagation** self-referencing (requires UNIQUE).
	1:N (or N:1)	- N side mandatory: **Key propagation** self-referencing (requires **NOT NULL**). - N side optional: **Key propagation** self-referencing. **Alternative: New table** to avoid NULL FKs.
	N:N	**New table** with two foreign keys referencing the primary key from the entity, and these two combined will form the primary key.
Binary	1:1	- Both sides mandatory: **Merge** into a single table if possible. - Both sides optional: **New table**. Alternative: Key propagation (requires **UNIQUE**). - One optional, One mandatory: **Key propagation** to optional entity (requires **UNIQUE**).
	1:N (or N:1)	- N side mandatory: **Key propagation** to the N side (requires NOT NULL). - N side optional: **Key propagation** to the N side. **Alternative: New table** to avoid NULL FKs.
	N:N	**New table** with two foreign keys referencing each entity, both forming a composite primary key.
Ternary	1:1:1	**New table**. All Foreign Keys combine to form the Primary Key.
	1:1:N	**New table**. Foreign Keys from both N sides are combined to form the Primary Key.
	1:N:N	**New table**. Only the Foreign Key from the N side as Primary Key.
	N:N:N	**New table**. Any one of the Foreign Keys can be the Primary Key.
Inheritance	**Key propagation**: the parent entity primary key is propagated to each descendant table as both a primary key and foreign key. (Same case as binary 1:1 One optional, One mandatory)	